Lab 2 · Redes neuronales recurrentes

Aprendizaje Computacional II · Parte 2 (Modelado de secuencias) · Juan Carlos Alcausa Luque · Noviembre de 2026

Cuadernos entregados: `L2_P1_nombres_resuelta.ipynb` y `L2_P2_siguiente_caracter_resuelta.ipynb` (PyTorch 2.15)

Resumen. *Parte I*: una RNN de 128 neuronas clasifica apellidos en 14 idiomas leyendo un carácter por paso. El código base acierta el 80,4 % del test; la versión mejorada (lotes empaquetados, validación, Adam y parada temprana) llega al 81,0 % y es unas 8 veces más rápida. El idioma está sobre todo en el sufijo, los errores van hacia English, y la RNN supera en unos 3 puntos a la mejor red feedforward. *Parte II*: el código de predicción del siguiente carácter de *The Time Machine* apenas entrenaba (10 pasos de gradiente) e inflaba la exactitud con un `<unk>` de 5 caracteres. Tras nueve experimentos, la configuración final obtiene **49,46 % ± 0,24 %** de exactitud y una **perplejidad de 5,96 ± 0,10** en test (20 ejecuciones).

Parte I · Predicción del origen de nombres

1. Codificación, red y estado oculto

Cada carácter se codifica en one-hot sobre un vocabulario de 58 símbolos (52 letras ASCII, 5 signos y «_» como desconocido), y un nombre de T letras es un tensor `(T, 1, 58)`. `nn.RNN(58, 128)` calcula $h_t = \tanh(x_t W_{xh} + h_{t-1} W_{hh} + b)$ desde $h_0 = 0$, con las mismas matrices en cada paso. Una capa lineal (128 → 14) y `LogSoftmax` convierten el **último** estado oculto en log-probabilidades de los 14 idiomas (25 870 parámetros). El estado oculto es la memoria de la red: un resumen de tamaño fijo de todo lo leído. Clasificando cada prefijo se ve cómo se forma: «N» → Russian (la clase mayoritaria), «Naka» → Japanese 0,94, «Ivanov» → Russian 1,00 al llegar la «v» final.

Nota: la guía habla de 18 clases, pero el `names.zip` proporcionado contiene 14 idiomas.

2. Evaluación, mejoras e hiperparámetros

Problemas del código base: `unicodeToAscii` se define pero no se usa (las letras acentuadas se codifican como «_»); los nombres se procesan de uno en uno (lento); no hay conjunto de validación; y `labels_uniq = list(set)` no es reproducible entre ejecuciones. Mejoras, medidas sobre el mismo test:

Paso	Exactitud	F1 ponderado	F1 macro	s / época
A · Base (nombre a nombre)	80,4 %	0,796	0,575	3,27
B · Lotes empaquetados	80,0 %	0,788	0,558	0,40
C · + unicodeToAscii	80,5 %	0,795	0,564	0,40
D · + Adam, media, parada temprana	81,0 %	0,801	0,578	0,37

El procesamiento por lotes con `pack_padded_sequence` da el mismo resultado unas 8 veces más rápido. La exactitud apenas cambia: la red base ya estaba cerca de su techo. Las ganancias son de velocidad, reproducibilidad y metodología. El clasificador trivial («siempre Russian») acierta el 47,7 %.

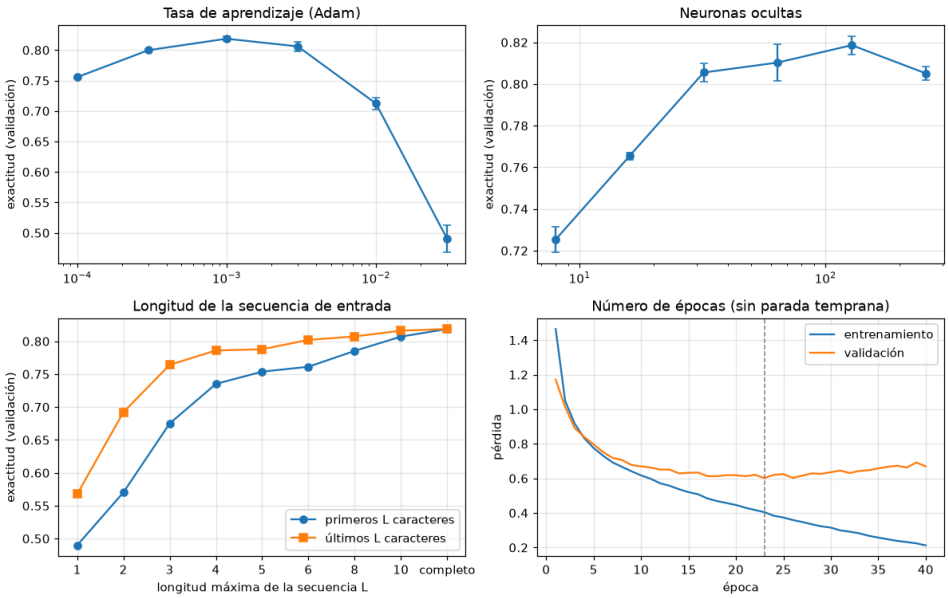


Figura 1. Hiperparámetros (exactitud en validación, media ± desv. típica de 3 semillas): tasa de aprendizaje, neuronas ocultas, longitud de la secuencia (primeros o últimos L caracteres) y curvas de 40 épocas sin parada temprana.

- **Tasa de aprendizaje** (Adam), la más sensible: óptimo en 0,001 (81,9 %); con 0,03 cae al 49 %, y con 0,0001 no converge en 40 épocas.
- **Neuronas ocultas**: de 72,5 % (8) a 81,9 % (128); con 256 algo peor (80,5 %).
- **Longitud de la secuencia**: con los últimos 6 caracteres ya acierta el 80,2 %, y con los primeros 6 solo el 76,1 %. El idioma está en el sufijo.
- **Épocas**: el mínimo de validación está en la época 23; en la 40, entrenamiento 93 % frente a validación 81 % (sobreajuste).

3. Matriz de confusión

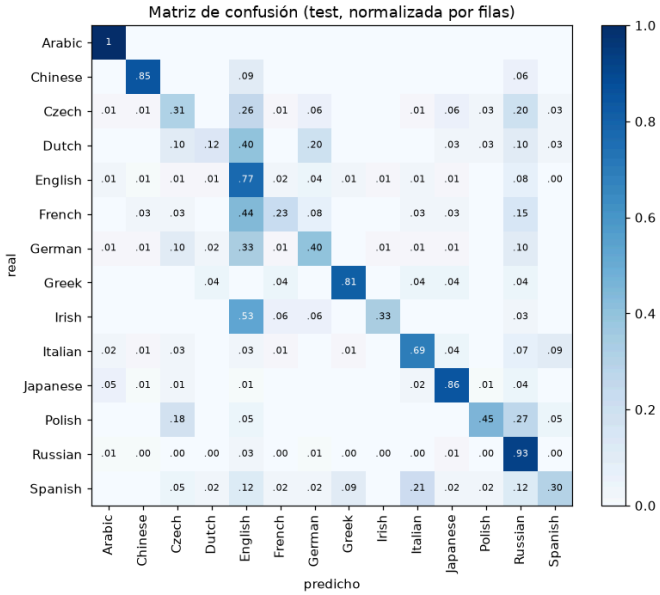


Figura 2. Matriz de confusión del modelo final en test, normalizada por filas.

Real → predicho	Nombres	% de la clase real
Russian → English	47	3,4 %
English → Russian	45	7,8 %
German → English	42	33,3 %
English → German	23	4,0 %
Czech → English	22	25,6 %
Irish → English	19	52,8 %
French → English	17	43,6 %
Czech → Russian	17	19,8 %

Casi todos los errores van hacia **English**: es la segunda clase más grande, y el archivo English contiene apellidos de origen alemán, francés, irlandés u holandés («Brandt», «Flynn»). Las clases con peor recall son las pequeñas: Dutch 12,5 %, French 23 %, Spanish 30 %, Czech 31 %. También hay confusiones dentro de las **familias lingüísticas** (Czech → Russian 20 %). El 3,9 % de los nombres de test aparece en varios idiomas («Abel», «Adam»), con una exactitud del 37,7 %: parte del error es irreducible. El 100 % de Arabic es **memorización**: sus 2000 líneas son solo 108 nombres distintos y todos los de test estaban en entrenamiento. Ponderar la pérdida por clase sube el recall de las clases pequeñas, pero baja la exactitud al 71,6 % y el F1 macro a 0,532.

4. Comparación con una red feedforward

Modelo (capa oculta de 128, 3 semillas)	Parámetros	Test	F1 macro
RNN (h=128)	25 870	81,4 % ± 0,7	0,580
FNN posicional izq. (h=128)	150 414	75,4 % ± 0,4	0,498
FNN posicional der. (h=128)	150 414	78,7 % ± 0,1	0,548
FNN bolsa de caracteres (h=128)	9 358	76,0 % ± 0,5	0,484

La RNN gana con 6 veces menos parámetros que las FNN posicionales, pero la diferencia es moderada porque los nombres son cortos y el idioma se decide en el sufijo. La FNN posicional tiene un peso distinto para cada posición: alinear el final del nombre (derecha) gana 3,2 puntos a alinear el principio. La bolsa de caracteres, sin orden, ya llega al 76 %.

Parte II · Predicción del siguiente carácter

1. El código original

Cada ejemplo es una ventana de 20 caracteres y el objetivo es el carácter 21. El modelo usa `nn.Embedding` → `nn.RNN` (ReLU) → dropout 0,5 sobre el último estado → capa lineal; la predicción es el argmax de los logits (`CrossEntropyLoss` incluye el softmax). Problemas detectados:

- **Solo 10 pasos de gradiente**: cada época procesa todo el conjunto como un único lote (`batch_size` no se usa), y la parada temprana nunca se activa (paciencia = épocas = 10).
- `<unk>` **de 5 caracteres**: el texto pasa de 178 979 a 214 451 caracteres y el 16,5 % son letras de `<unk>`, triviales de predecir.
- `nn.Embedding` **en lugar de one-hot**, en contra del enunciado.

El original obtiene entre el 10 % y el 37,7 % según el tamaño oculto. Con el texto corregido (un único «_» como desconocido, saltos de línea como espacio, 28 símbolos) y one-hot, la misma configuración se queda en el **20,6 %**, frente al 18,3 % de predecir siempre un espacio.

2. Experimentos de mejora

Proceso acumulativo: cada familia de opciones se evalúa con 3 semillas en validación sobre la mejor configuración hasta ese momento, y la ganadora pasa al siguiente experimento. Se entrena con minilotes y parada temprana (paciencia 5) y se evalúa siempre el carácter 21.

Experimento	Exactitud en validación (media de 3 semillas)	Rango (puntos)
1 · Lotes y pérdida	P0 · original corregido (lote completo, 10 épocas): 18,7 % · P1 · + minilotes de 128 y parada temprana: 40,9 % · **P2 · + pérdida en las 20 posiciones: 44,7 %**	26,0
2 · Tasa (Adam)	0.0003: 42,7 % · 0.001: 43,9 % · 0.003: 44,3 % · **0.01: 44,7 %** · 0.03: 41,3 %	3,4
3 · Optimizador	SGD (lr 1): 44,3 % · SGD + momento 0,9 (lr 0,1): 44,4 % · RMSprop (lr 1e-3): 43,8 % · **Adam (lr 0.01): 44,7 %** · AdamW (lr 0.01, wd 0,01): 44,4 %	0,9
4 · Activación y dropout	**tanh · dropout 0.0: 47,4 %** · tanh · dropout 0.2: 46,1 % · tanh · dropout 0.5: 44,0 % · relu · dropout 0.0: 47,0 % · relu · dropout 0.2: 46,5 % · relu · dropout 0.5: 44,7 %	3,4
5 · Inicialización	defecto (uniforme ±1/√h): 47,6 % · Xavier: 47,4 % · ortogonal (W_hh): 47,4 % · **He (Kaiming): 47,7 %**	0,3
6 · Neuronas ocultas	16: 40,6 % · 32: 44,9 % · 64: 47,7 % · 128: 47,8 % · **256: 48,2 %** · 512: 36,6 %	11,6
7 · Dropout (h = 256)	**0.0: 48,2 %** · 0.2: 47,8 % · 0.3: 47,1 % · 0.5: 46,0 %	2,2
8 · Recorte del gradiente	sin recorte: 47,4 % · **norma 0,25: 48,2 %** · norma 1: 48,2 % · norma 5: 47,5 %	0,8
9 · Tasa variable	constante: 48,2 % · **se reduce a la mitad si la validación no mejora: 49,2 %**	1,0

- **Minilotes** (+22 puntos) y **pérdida en las 20 posiciones** (+3,8) son los cambios decisivos.
- **Quitar el dropout del 50 %** mejora unos 3 puntos; la parada temprana regulariza lo suficiente.
- **Tamaño oculto**: mejora hasta 256; con 512 y tasa 0,01 el entrenamiento es inestable.
- **Reducir la tasa a la mitad** cuando la validación no mejora añade 1 punto: con tasa 0,01 constante la pérdida de entrenamiento acababa subiendo.
- **Optimizador, inicialización y recorte** influyen menos de 1 punto.
- **Gradientes**: con tanh y 20 pasos no explotan (norma ≤ 3). Con ReLU, una tasa 10 veces mayor y sin recorte, la norma se hace infinita y los pesos acaban en NaN; con recorte a 1 la red sobrevive (40 %).

Configuración final: h = 256, tanh, sin dropout, inicialización He, Adam con tasa 0,01 que se reduce a la mitad si la validación no mejora, recorte a 0,25, pérdida en las 20 posiciones, lotes de 128 y parada temprana. En test (semilla 0): 49,8 % y perplejidad 5,93, frente a 20,6 % y 17,4 del original corregido.

3. Experimento final: 20 ejecuciones

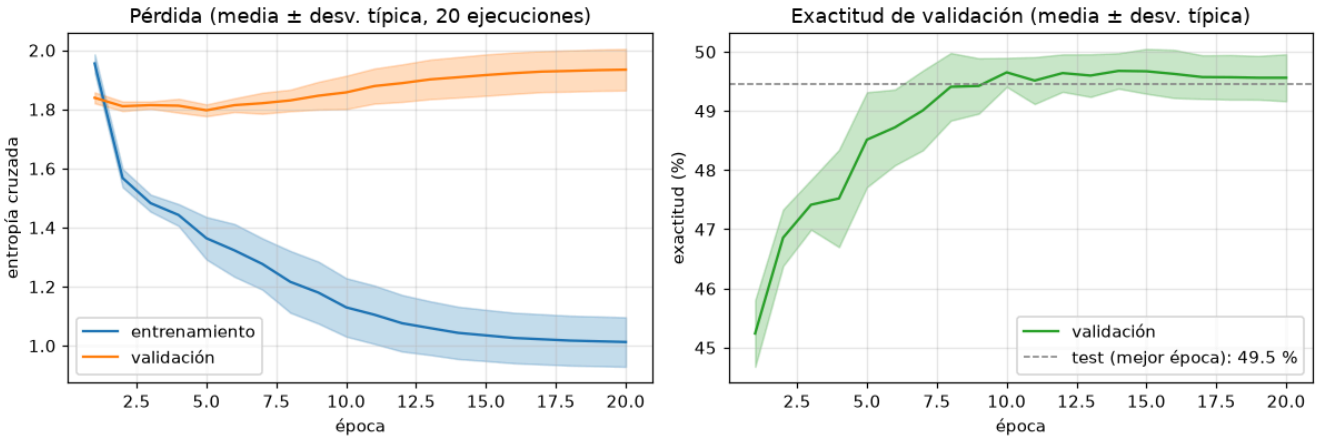


Figura 3. Curvas de aprendizaje medias ± desviación típica de 20 ejecuciones con semillas distintas (20 épocas; en test se evalúa el modelo con menor pérdida de validación).

Exactitud en test: 49,46 % ± 0,24 % (media ± desviación típica; mínimo 49,0 %, máximo 50,0 %). La pérdida de entrenamiento baja de forma continua; la de validación es mínima hacia la época 5 y luego sube: con ≈ 53 000 caracteres de entrenamiento y ≈ 80 000 parámetros la red sobreajusta pronto, y la parada temprana (mejor época entre la 4 y la 10) es imprescindible. La exactitud de validación sigue subiendo ligeramente aunque la pérdida suba: la red se vuelve demasiado segura en sus errores.

4. Perplejidad

Perplejidad en test: 5,964 ± 0,098: la red duda en promedio entre unos 6 caracteres (uniforme: 28). Comparación con modelos de n-gramas por conteo (suavizado de Laplace) entrenados con el mismo 30 % del texto:

Modelo	Perplejidad	Exactitud
1-grama (contexto 0)	17,80	18,3 %
2-grama (contexto 1)	10,26	29,2 %
3-grama (contexto 2)	7,28	39,5 %
4-grama (contexto 3)	7,37	48,1 %
5-grama (contexto 4)	9,81	49,8 %
6-grama (contexto 5)	13,38	45,7 %
RNN vanilla (contexto 20), 20 ejecuciones	5,96	49,5 %

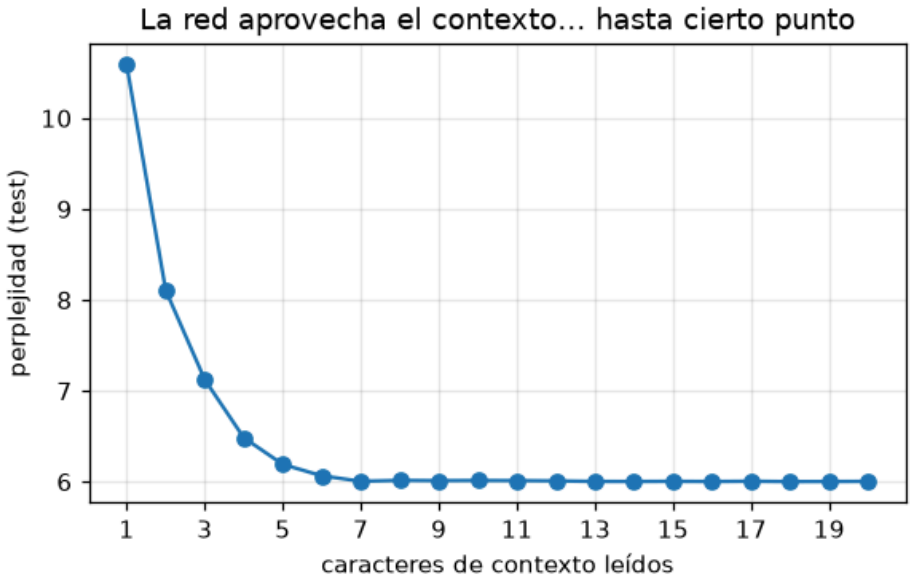


Figura 4. Perplejidad en test según los caracteres de contexto leídos dentro de la ventana.

La RNN tiene la mejor perplejidad (probabilidades mejor calibradas) y en exactitud empata prácticamente con el 5-grama, que con tan poco texto es una regla muy buena para el carácter más probable. La perplejidad se estanca a partir de unos 7 caracteres de contexto: la memoria efectiva de la RNN vanilla es corta. El texto generado tiene palabras correctas pero ninguna coherencia global, y la generación voraz entra en bucles.

Conclusiones

- **Parte I**: la RNN resume el nombre en su último estado oculto y alcanza el 81,0 % (trivial: 47,7 %). Las mejoras más útiles son de código (lotes empaquetados, unas 8 veces más rápido) y de metodología (validación y parada temprana). La tasa de aprendizaje es el hiperparámetro más sensible. El idioma se decide en el sufijo. Los errores se deben al desequilibrio (hacia English), a las familias lingüísticas y a nombres compartidos. La RNN supera a la mejor FNN en unos 3 puntos con muchos menos parámetros.
- **Parte II**: lo que más influye es entrenar de verdad (minilotes, ≈ 26 puntos junto con la pérdida en todas las posiciones), después el tamaño oculto (≈ 12), la tasa de aprendizaje y el dropout (≈ 3 cada uno) y la reducción de la tasa durante el entrenamiento (1). El optimizador, la inicialización y el recorte influyen menos de 1 punto en exactitud, aunque el recorte evita la divergencia en configuraciones arriesgadas. Resultado final: 49,46 % ± 0,24 % y perplejidad 5,96.
- Las limitaciones observadas (sobreajuste rápido, memoria de unos 5–7 caracteres, texto sin coherencia global) son las que la teoría atribuye a las RNN vanilla y las que motivan las LSTM y GRU.